

Generatore di guida Bash da esame (Ctrl+F oriented, versione estesa)

Questa guida è costruita per essere **cercata con Ctrl+F** partendo da: - parole "da traccia" (spesso imprecise) - sinonimi naturali ("contare righe", "trovare file", "stampare solo numero", "invertire", "terza parola") - pattern ricorrenti visti negli esercizi del progetto

Vincolo operativo: **niente AWK**.

1) Indice esteso per Ctrl+F (Problema / formulazione → comando/i)

A. Leggere input, righe, parole, caratteri

- leggere riga per riga da stdin → `while read ...; do ...; done`
- leggere riga per riga da file → `while read ...; do ...; done < file`
- leggere un file che NON finisce con newline (ultima riga senza \n) → `while read ... || [[ "${RIGA}" != "" ]]; do ...; done`
- leggere una sola parola per riga → `read VAR` (con input senza spazi) oppure `read A B C ...`
- leggere campi separati da spazi → `read NOME COGNOME MATRICOLA VOTO`
- leggere terza parola di ogni riga → `read A B C D; echo "${C}"`
- leggere carattere per carattere → `read -N 1 -r CH`
- leggere N caratteri esatti → `read -N N ...`
- leggere al massimo N caratteri → `read -n N ...`
- non interpretare backslash → `read -r ...`

Parole chiave da Ctrl+F: `read`, `while read`, `-N`, `-n`, `-r`, ultima riga, EOF, carattere per carattere, terza parola, campi.

B. Redirezione, pipe, stderr, file descriptor

- passare un file a uno script (stdin da file) → `./script.sh < file`
- passare output di un comando a uno script → `cat file | ./script.sh`
- scrivere su stderr → `echo "... " 1>&2` oppure `echo "... " >&2`
- salvare solo il numero di righe (senza nome file) → `wc -l < file`
- aprire file con exec e leggere con file descriptor → `exec {FD}< file + read -u ${FD} ...`
- scrivere su un FD aperto → `echo "... " 1>&${FD}`
- chiudere FD → `exec {FD}>&-` (o `exec n<&-`)
- applicare redirezione a un intero blocco → `while ...; do ...; done > out.txt`
- duplica/autoridireziona stdin → `exec 0>&${NUOVOFD}` (scenario avanzato)

Parole chiave: `pipe`, `|`, `stderr`, `2>`, `1>&2`, `wc -l <`, `exec {FD}<`, `read -u`.

C. Cercare file/directory e filtrare

- cercare file per nome → `find PATH -name "pattern"`
- cercare SOLO nella directory, non nelle sottodirectory → `find PATH -maxdepth 1 -name ...`
- trovare solo directory → `find PATH -type d`
- trovare solo file → `find PATH -type f`
- escludere una directory dalla ricerca → `find . -path ./ESAMI -prune -o ...`
- cercare testo dentro file → `grep "stringa" file`
- cercare testo dentro più file trovati da find → `find ... -exec grep -H ... '{}' \;`

Parole chiave: `find`, `-name`, `-type`, `-maxdepth`, `-exec`, `grep -H`, `-prune`.

D. Filtrare righe/contare

- contare righe → `wc -l`
- contare caratteri → `wc -c` (attenzione: include newline se presente)
- contare caratteri senza contare newline finale (per riga) → dipende dal modo in cui stampi; vedi sezione `wc`
- righe che contengono un pattern → `grep "pattern" file`
- righe che NON contengono un pattern → `grep -v "pattern" file`

Parole chiave: `wc -l`, `wc -c`, `grep -v`, `contare`.

E. Estrarre pezzi: caratteri, campi, sottostringhe

- primi 3 caratteri di una riga → `cut -b 1-3` (byte) oppure `cut -c 1-3` (caratteri)
- colonna N con delimitatore → `cut -d ':' -f N`
- dalla colonna N fino alla fine → `cut -d ' ' -f2-`
- estrarre 1 carattere dalla variabile (posizione COUNT) → `echo "${VAR:${COUNT}:1}"`
- sostituire testo in variabile senza sed → `${OUT//gatto/cane}`

Parole chiave: `cut -d`, `-f`, `-f2-`, `cut -c`, `cut -b`, `substring`, `${VAR:...}`.

F. Ordinare output

- ordinare righe → `sort`
- ordinare per campo K → `sort -k 3` (es: cognome in terzo campo)
- trucco: aggiungi campo davanti per ordinare, poi taglia → `echo "$COGNOME ..." | sort | cut -d ' ' -f2-`

Parole chiave: `sort`, `sort -k`, `ordinare`, `campo`.

G. Argomenti, spazi, quoting

- passare tutti gli argomenti mantenendo i confini → `"$@"`
- iterare su argomenti correttamente → `for x in "$@"; do ...; done`

- **file con spazi nel nome** → sempre `"$var"` e `"nome con spazio.txt"`
- **perché non funziona mettere tanti nomi in una variabile** → perdi confini (vedi esercizio)

Parole chiave: `"$@"`, `spazi`, `quoting`, `argomenti`.

H. Sequenze condizionali, exit status, errori tipici

- **esegui B solo se A riesce** → `A && B`
- **esegui B solo se A fallisce** → `A || B`
- **sequenza normale** → `A ; B ; C`
- **controllo dell'ultimo exit code** → `$?`
- **confronto numerico** → `(( ... ))`
- **test su file esistenza** → `[[ -e file ]]`

Parole chiave: `&&`, `||`, `$?`, `exit status`, `(( ))`, `[[ ]]`.

I. Processi e segnali (se compaiono in tracce)

- **vedere processi** → `ps`
- **mettere in background** → `&`
- **aspettare figli** → `wait`
- **sleep** → `sleep N`
- **uccidere processo** → `kill PID` / `killall nome`
- **job control** → `bg`, `fg`

Parole chiave: `ps`, `kill`, `killall`, `&`, `wait`, `sleep`, `bg`, `fg`.

2) Sezione comandi (schede operative, estese)

2.1 `read` (fondamentale)

Problema che risolve: leggere input da stdin o da un file/pipe **senza usare comandi esterni**. È il cuore di quasi tutti gli esercizi.

Parole chiave / Ctrl+F: `leggere`, `input`, `riga`, `EOF`, `parola`, `campi`, `carattere`, `-N`, `-n`, `-r`, `-u`.

Sintassi essenziale: - Riga intera: `read RIGA` - Più campi: `read A B C D` (splitta per spazi/tab; l'ultimo prende "il resto" se metti meno variabili) - Da file descriptor: `read -u ${FD} VAR...`

Opzioni d'esame da ricordare: - `-r` → non interpreta `\` come escape (quasi sempre giusto in file reali) - `-N 1` → legge **1 carattere** (include spazi e newline) - `-N N` → legge **esattamente N** caratteri (se trova newline, lo legge come carattere e continua) - `-n N` → legge **fino a N** caratteri ma si ferma a newline

Pattern chiave 1: leggere righe fino a EOF (stdin)

```
while read RIGA ; do
    echo "${RIGA}"
done
```

Pattern chiave 2: leggere righe da file (redirezione del blocco)

```
while read RIGA ; do
    echo "${RIGA}"
done < input.txt
```

Pattern chiave 3: file senza newline finale (ultima riga non persa)

```
while read RIGA || [[ "${RIGA}" != "" ]] ; do
    echo "${RIGA}"
done < miofileNoNL.txt
```

Pattern chiave 4: terza parola di ogni riga

```
while read A B C D ; do
    echo "${C}"
done < /usr/include/stdio.h
```

Se la terza parola non esiste, `C` è vuota → stampi riga vuota (o niente se fai un controllo).

Pattern chiave 5: carattere per carattere

```
NUM=0
while read -N 1 -r CH ; do
    ((NUM=NUM+1))
done < /usr/include/stdio.h
echo "${NUM}"
```

Errori tipici d'esame: - Dimenticare `-r` e poi "spariscono" backslash. - Usare `read` dentro una pipe e poi pretendere che una variabile sopravviva fuori (in alcune shell/pipeline crea subshell). Se vuoi sicurezza, preferisci `while ...; do ...; done < file` quando puoi. - Confondere `-N` e `-n` (sono diversi, soprattutto col newline).

2.2 `exec` (file descriptor + I/O avanzato)

Problema: aprire file in lettura/scrittura **senza usare cat** e gestire più input contemporanei.

Parole chiave: `file descriptor`, `FD`, `exec {FD}<`, `read -u`, `chiudere FD`, `stream`.

Sintassi essenziale (consigliata): - Solo lettura: `exec {FD}< percorsoFile` - Solo scrittura: `exec {FD}> percorsoFile` - Append: `exec {FD}>> percorsoFile` - Lettura+scrittura: `exec {FD}<> percorsoFile` - Chiudere: `exec {FD}>&-`

Esempio d'esame: leggere da un file con FD

```
exec {FD}< /usr/include/stdio.h
if (( $? == 0 )) ; then
  while read -u ${FD} A B C D ; do
    echo "${C}"
  done
  exec {FD}>&-
fi
```

Quando usarlo davvero in esame: - quando ti chiedono esplicitamente FD/exec - quando devi leggere da file ma vuoi evitare pipeline (più controllabile)

Errori tipici: - dimenticare di chiudere FD (`exec {FD}>&-`) - mettere spazi sbagliati: gli operatori sono attaccati a numero o a `{VAR}`.

2.3 Redirezione e pipe (`<`, `>`, `>>`, `|`, `1>&2`, ecc.)

Problema: collegare comandi e controllare input/output.

Parole chiave: `pipe`, `stdin`, `stdout`, `stderr`, `reindirizzare`, `scrivere su file`, `append`, `solo numero`.

Essenziale: - `cmd > out.txt` → stdout su file (sovrascrive) - `cmd >> out.txt` → stdout su file (append)
- `cmd < in.txt` → stdin dal file - `cmd1 | cmd2` → stdout di cmd1 diventa stdin di cmd2 - `echo "x" 1>&2` → scrive su stderr

Trucco da esame: "voglio solo il numero, non il nome file" - `wc -l file` produce: `N file` - `wc -l < file` produce: `N`

Redirezione su un intero blocco (molto utile)

```
while read RIGA ; do
    echo "${RIGA}"
done < input.txt > output.txt
```

2.4 `wc`

Problema: contare righe, parole, caratteri.

Parole chiave: `contare`, `numero righe`, `numero caratteri`, `solo numero`.

Uso tipico: - `wc -l file` → righe (ma stampa anche nome file) - `wc -l < file` → solo numero - `wc -c file` → byte (include newline se presente)

Errore tipico: - "mi conta anche il newline": sì, `wc -c` conta byte, quindi anche `\n`.

2.5 `cut`

Problema: estrarre campi (delimitati) o porzioni di caratteri.

Parole chiave: `prima/seconda/terza colonna`, `delimitatore`, `PATH`, `:`, `primi 3 caratteri`, `colonne`.

Sintassi che devi sapere sotto pressione: - Per caratteri: `cut -c 1-3` (caratteri) - Per byte: `cut -b 1-3` (byte) - Per campi: `cut -d ':' -f 1` (delimitatore `:`) - Dal campo N in poi: `cut -d ' ' -f2-`

Esempio pattern: separare PATH (senza AWK)

```
# stampa ogni percorso (field) su una riga
# idea: trasformo ':' in '\n' e poi gestisco per righe
# (se tr è disponibile nei tuoi comandi; se non lo vuoi usare, vedi pattern
alternativi)
```

Se nei tuoi vincoli non vuoi usare `tr`, usa `read` con IFS (vedi pattern in sezione 3).

Esempio pattern: eliminare primo campo dopo sort

```
echo "${COGNOME} ${MATRICOLA} ${NOME} ${COGNOME} ${VOTO}"
| sort
| cut -d ' ' -f2-
```

Errori tipici: - `-d` prende **un singolo carattere** come delimitatore. - Se ci sono spazi multipli, `cut -d ' ' -f...` può creare campi vuoti; spesso meglio normalizzare a monte (se ti è permesso) o leggere con `read`.

2.6 `grep`

Problema: filtrare righe che contengono (o non contengono) un pattern.

Parole chiave: `contiene`, `non contiene`, `asterisco`, `filtrare righe`, `cercare stringa`.

Sintassi essenziale: - `grep 'pattern' file` - `grep -v 'pattern' file` (negazione) - `grep -H 'pattern' file` (stampa anche nome file; utile con find)

Pattern d'esame da memorizzare: - "righe di `stdio.h` che contengono " → `grep '*' /usr/include/stdio.h` - "righe che **NON** contengono " → `grep -v '*' /usr/include/stdio.h`

Errori tipici: - pattern che include caratteri speciali: se sei in dubbio, mettilo tra apici singoli.

2.7 `sort`

Problema: ordinare output.

Parole chiave: `ordinare`, `ordine crescente`, `per cognome`, `per campo`.

Sintassi essenziale: - `sort` (ordine lessicografico) - `sort -k 3` (ordina per terzo campo)

Pattern reale (esercizio studenti): - produci righe con `while read ...; do ...; done < file` e poi mandali in pipe a `sort -k ...`

2.8 `find`

Problema: cercare file/directory e applicare un comando su ciascuno.

Parole chiave: `cercare`, `file`, `directory`, `sottodirectory`, `ricorsivo`, `solo livello`, `-exec`.

Sintassi essenziale: - `find /usr/include -name "*std*.h"` - `find ./BUTTAMI -type d` - `find PATH -maxdepth 1 -name "pattern"`

Pattern: applicare `grep` ai file trovati

```
find . -type f -name "*.c" -exec grep -H rand '{}' \;
```

Pattern: escludere directory (prune)

```
find ./ -path ./BUTTA -prune -o -type f -name "*.c" -exec grep -H rand '{}' \;  
| more
```

Errori tipici: - path sbagliato (typo): `/usr/include` non esiste. - dimenticare le virgolette su pattern con `*`.

2.9 `sed` (quando serve davvero)

Problema: sostituire stringhe in output.

Parole chiave: `sostituire`, `replace`, `s/vecchio/nuovo/`.

Nota pratica: se devi sostituire dentro una **variabile**, spesso è più veloce usare la parameter expansion: - `${OUT//gatto/cane}`

Se invece stai lavorando in pipeline su testo:

```
echo "${OUT}" | sed 's/gatto/cane/g'
```

2.10 Argomenti e quoting (`$#`, `$1`, `"$@"`)

Problema: passare parametri senza perdere i confini, soprattutto con spazi.

Parole chiave: `argomenti`, `parametri`, `spazi`, `confini`, `$@`, `$#`.

Regole da esame (da applicare sempre): - Dentro uno script, se vuoi iterare sugli argomenti:

```
for name in "$@" ; do  
    echo "$name"  
done
```

- Se vuoi passare *tutti* gli argomenti a un altro script mantenendo i confini:


```
./altro.sh "$@"
```

Errore tipico (esercizio con variabile d'ambiente NOMIFILES): - Mettere tanti nomi in una stringa e poi iterare con `for` → perde confini quando ci sono spazi.

2.11 `echo` (e `stdout/stderr`)

Problema: stampare variabili e output.

Parole chiave: `stampare`, `output`, `stderr`.

Pattern chiave: - `stdout`: `echo "${VAR}"` - `stderr`: `echo "${PAROLA}" 1>&2`

2.12 Controllo di flusso: `if`, `while`, `for`, `&&`, `||`, `(( ))`, `[[ ]]`

Problema: trasformare una traccia in algoritmo robusto.

Parole chiave: `condizione`, `se`, `finché`, `loop`, `confronto`, `exit code`.

Template "standard d'esame" (robusto e corto):

```
#!/bin/bash

# validazione argomenti (se richiesta)
if (( "$#" != "2" )) ; then
    echo "usage: ..." 1>&2
    exit 1
fi

# loop su righe di un file
while read A B C ; do
    if (( ... )) ; then
        echo "..."
    fi
done < "$1"
```

Sequenze condizionali: - `A && B` (esegue B se A ok) - `A || B` (esegue B se A fallisce)

Errori tipici: - Confondere `[[ ... ]]` con `(( ... ))` (uno è test/logico su stringhe/file, l'altro è aritmetico)

2.13 Processi: `ps`, `sleep`, `&`, `wait`, `kill`, `killall`, `bg`, `fg`

Problema: tracce su processi, concorrenti, segnali.

Parole chiave: `pid`, `processo`, `background`, `attendere`, `terminare`.

Pattern base:

```
./discendenti.sh $((N-1)) &  
wait
```

Nota: `ps` è tipicamente usato per “vedere cosa succede” (dimostrazioni / debug).

3) Sezione pattern d’esame (problemi ricorrenti → soluzioni tipiche)

Pattern 1 — “Leggo righe da stdin fino a EOF”

Traccia tipica: “lo script legge righe dallo standard input fino a EOF”

```
while read RIGA ; do  
    # usa ${RIGA}  
done
```

Se la riga può contenere backslash: `while read -r RIGA ; do ...; done`

Pattern 2 — “Leggo parole da ogni riga”

Traccia: “le righe sono composte da parole separate da spazi”

```
while read P1 P2 P3 RESTO ; do  
    echo "${P1}"  
    # oppure ${P3}, ecc.  
done
```

Pattern 3 — “Estraggo la prima parola e la mando su stderr”

Traccia: “estrarre la prima parola e scriverla su stderr”

```
while read P1 RESTO ; do
    echo "${P1}" 1>&2
done
```

Pattern 4 — “Conto righe / voglio solo il numero”

- `wc -l < file`

Pattern 5 — “Conto caratteri letti dal file (carattere per carattere)”

```
NUM=0
while read -N 1 -r CH ; do
    ((NUM=NUM+1))
done < file

echo "${NUM}"
```

Pattern 6 — “Ultima riga senza newline finale”

```
while read -r RIGA || [[ "${RIGA}" != "" ]] ; do
    echo "${RIGA}"
done < file
```

Pattern 7 — “Filtro righe che contengono / non contengono”

- contiene: `grep 'X' file`
- non contiene: `grep -v 'X' file`

Pattern 8 — “Ordino output prodotto da un while”

```
while read ... ; do
    echo "..."
done < input.txt | sort -k 3
```

Pattern 9 — “Join logico tra due file tramite chiave (matricola) senza strutture dati”

Idea: per ogni riga del file2: 1) controllo condizione (es. voto < 18) 2) cerco la chiave nel file1 con `grep` e conto con `wc -l` 3) se zero → stampo 4) alla fine ordino

Template:

```
while read NOME COGNOME MATRICOLA VOTO ; do
  if (( VOTO < 18 )) ; then
    LINES=`grep ${MATRICOLA} RisultatiProvaPratica1.txt | wc -l`
    if [[ "${LINES}" == "0" ]] ; then
      echo "${MATRICOLA} ${NOME} ${COGNOME} ${VOTO}"
    fi
  fi
done < RisultatiProvaPratica2.txt | sort -k 3
```

Pattern 10 — “Trucco: aggiungi campo davanti per ordinare, poi taglia via”

Se vuoi ordinare per cognome ma poi output diverso: 1) stampa prima il cognome 2) `sort` 3) `cut -d ' ' -f2-`

4) Sezione esercizi del progetto (esercizio → comandi → schema risolutivo)

Esercizi su quoting e metacaratteri (directory BUTTAMI)

Traccia: creare file con nomi speciali (`*`, `**`, `***`, `;;`), poi aggiungere `.txt`, copiare `/usr/include`, listare sottodirectory, eliminare `include` copiato.

Comandi coinvolti: `mkdir`, `touch`, `ls`, `for`, `cp -R`, `find -type d`, `rm -rf`.

Schema risolutivo: - crea directory - crea file usando quoting (`BUTTAMI/"**"`) - ciclo `for i in BUTTAMI/*; do touch "$i.txt"; done` - copia ricorsiva `cp -R` - `find` per directory - rimozione sicura `rm -rf BUTTAMI/include`

Errore tipico: usare command substitution `for i in `ls ...`` e rompere tutto coi caratteri speciali.

Esercizio “leggere solo terza parola di stdio.h”

Comandi: `exec {FD}<`, `read -u`, `echo`, `while`.

Schema: - apri FD - loop su `read -u` con 4 variabili - stampa `${C}` - chiudi FD

Esercizio “leggere caratteri uno a uno e contare”

Comandi: `exec`, `read -N 1 -r`, aritmetica `(( ))`, `echo`.

Schema: - apri FD - loop `read -N 1` incrementa contatore - chiudi FD - stampa contatore

Esercizio “file senza newline finale” (leggitutto)

Comandi: `exec`, `read -u`, `[[ ]]`, `echo`.

Schema: - apri FD in lettura - loop con condizione “EOF oppure riga non vuota” - stampa ogni riga - chiudi FD

Esercizio studenti (insuff2)

Comandi: `while read`, `grep`, `wc -l`, `sort -k`, `cut` (nella variante).

Schema: - loop su prova2 - filtra voto < 18 - verifica assenza in prova1 - stampa campi - ordina per cognome

Errori tipici: - confronti numerici fatti come stringhe - grep non quotato quando la chiave può contenere caratteri strani (qui matricola numerica: ok)

Esercizi “separare PATH” (senza AWK)

Obiettivo tipico: “stampare ogni path e la sua lunghezza”

Soluzione concettuale (solo built-in): usare `IFS=:` e `read`.

Schema: - salva IFS - imposta `IFS=:` - usa `read -a` sarebbe comodo ma se non è nel tuo materiale, allora: - usa un loop che “consuma” a pezzi con parameter expansion (vedi sezione `${VAR:...}`) oppure - usa pipeline con strumenti consentiti se presenti.

(Questa parte la rifiniamo quando mi confermi quali varianti di `read` e quali strumenti oltre a `cut` sono effettivamente nel tuo elenco consentito.)

5) Mini-sezione “confusioni tipiche da esame” (da Ctrl+F)

`./script.sh < file` VS `cat file | ./script.sh`

- entrambi danno allo script lo stesso **contenuto su stdin**.
- differenza pratica: la seconda lancia `cat` in più (più processi, più rumore). Se non ti serve, preferisci `< file`.

`wc -l file` VS `wc -l < file`

- il primo stampa anche il nome file.

- il secondo stampa solo il numero.

`read -n` VS `read -N`

- `-n N`: legge fino a N caratteri ma si ferma su newline.
- `-N N`: legge esattamente N caratteri, newline incluso.

`for i in $(ls ...)` (command substitution) VS `for i in DIR/*`

- con `$(ls ...)` perdi robustezza: spazi, caratteri strani.
- con `DIR/*` mantieni correttamente i nomi.

6) Cheat-sheet finale ultra-rapido (da stampare)

- Leggere righe stdin:
 - `while read -r R; do ...; done`
- Leggere righe file:
 - `while read -r R; do ...; done < file`
- Ultima riga senza newline:
 - `while read -r R || [[ "$R" != "" ]]; do ...; done < file`
- Terza parola:
 - `while read A B C D; do echo "$C"; done < file`
- Carattere per carattere:
 - `while read -N 1 -r C; do ((N++)); done < file; echo $N`
- Solo numero righe:
 - `wc -l < file`
- Contiene / non contiene:
 - `grep 'X' file` / `grep -v 'X' file`
- Find per nome:
 - `find PATH -name "pattern"`
- Find solo directory:
 - `find PATH -type d`
- Find + grep su *.c:
 - `find . -type f -name "*.c" -exec grep -H rand '{}' \;`
- Ordinare per campo 3:
 - `... | sort -k 3`
- Passare argomenti robusto:
 - `./x.sh "$@"` e `for a in "$@"; do ...; done`

Una domanda critica (una sola)

Se domani il prof ti vietasse improvvisamente **sed** o **grep** (ma lasciasse `read`, `cut`, `find`, `sort`), quali esercizi del tuo esame diventerebbero subito "a rischio" e quali invece riusciresti a fare lo stesso?